GRC102 Practical Lab 8: Monitoring, Auditing Controls and Executive Reporting

Khadija Nafale Likoro

C11_CGRCE_17281 (GRC102, Cohort 11)

Repository URL: https://github.com/Khadijanafalelikoro/GRC102-MONITTORING-AUDITING-AND-REPORTING.GIT

Safety statement: all work was done in a local Kali Linux VM with Docker. No public, organisational or third-party system was scanned. All customer data is fictitious and all passwords are lab-only values.

Part 1: Setting Up a Simulated Environment

1.1 Environment build

The environment has three containers on a Docker network: a vulnerable Apache Struts2 web server, a MySQL 5.7 database holding fictitious customer records, and a monitoring host. Docker was installed and tested with hello-world, the network secgov_network was created, and the stack was started with docker compose.

```
docker network create secgov_network
docker compose up -d
docker ps
```

```
┌──(icdfa㉿icdfa)-[~/security_governance_lab]
└─$ docker network ls
NETWORK ID     NAME            DRIVER    SCOPE
17e626206025   bridge          bridge    local
642fb763058c   host            host      local
bb446d42e53d   none            null      local
f6081374c097   secgov_network  bridge    local
```

```
✓ Container web_server         Started
✓ Container database_server    Started
✓ Container monitoring_server  Started
sudo] password for icdfa:
ONTAINER ID   IMAGE                  COMMAND                  CREATED         STATUS         PORTS
24f541ba3ed   vulhub/struts2:2.3.30  "/usr/local/bin/mvn-…"   32 seconds ago  Up 30 seconds  0.0.0.0
fce8b04e84c   ubuntu:20.04           "tail -f /dev/null"      32 seconds ago  Up 30 seconds
```

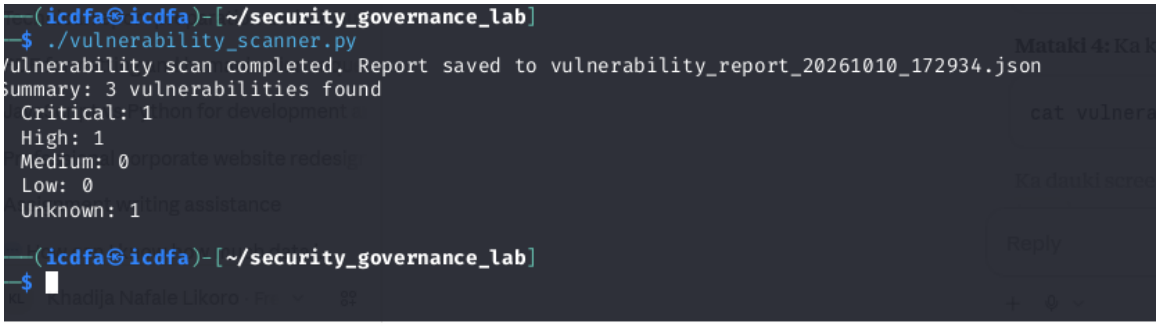
1.2 Problems found and fixed

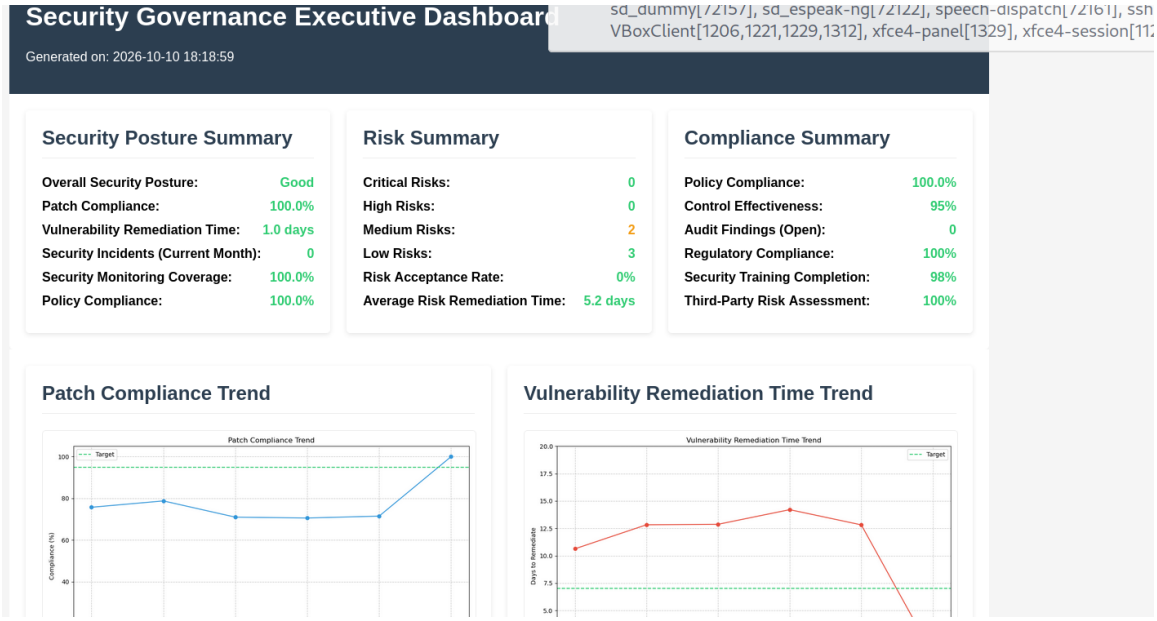
- The lab image tag vulhub/struts2:s2-045 does not exist (manifest unknown). The official Vulhub repository uses vulhub/struts2:2.3.30, which I used instead.
- The first docker-compose.yml failed with a YAML error on a mistyped line; I rebuilt it with correct indentation and validated it with docker compose config.
- The database container exited with ERROR 1064 because init.sql had typos (table name and missing KEY). I removed the container, rewrote init.sql and restarted it. The three fictitious rows loaded correctly.
- Extracted scripts had a leading indent that caused IndentationError; I removed it with a short script and all eight scripts compiled.

1.3 Vulnerability scan and governance tracker

```
./initialize_governance.py
./vulnerability_scanner.py
./governance_tracker.py --dashboard
```

The baseline scan found Apache Struts2 S2-045 (CVE-2017-5638) as Critical and MySQL using default credentials as High. The segmentation check first returned Unknown with a utf-8 decoding error, because the MySQL handshake is binary; I fixed it by decoding with errors="replace", and the rerun reported Medium. The tracker was initialised with 3 policies, 3 controls, 3 risks, 1 incident and 3 metrics.





1.4 How the environment reflects the Equifax conditions

Equifax was breached through a known Apache Struts flaw (CVE-2017-5638) that stayed unpatched after a fix was available. My lab runs the same vulnerable component on its public-facing tier. The database is reachable with a default password and shares a flat network with the web server, so one compromised tier exposes the data tier. Nothing monitors or alerts on the attack.

The tracker holds patch and vulnerability policies and a patch compliance metric below target, which matches the Equifax pattern where policies existed on paper but were not enforced. The environment therefore reproduces both the technical weaknesses and the governance weaknesses that led to the breach.

Part 2: Analysing Security Governance Failures

2.1 Attack simulation

```
./simulate_attack.py
cat attack_log.json
```

The simulation ran only against my own local container. It confirmed the Struts vulnerability, simulated database access and data exfiltration, and wrote attack_log.json. No exploit was run against the data.

```
$ ./simulate_attack.py
imulating attack on vulnerable Struts application...
ystem is vulnerable to Struts S2-045 (CVE-2017-5638)
imulating data exfiltration...
imulating access to database...
imulating exfiltration of customer data...
ttack simulation completed. Log saved to attack_log.json

(icdfa㉿icdfa)-[~/security_governance_lab]
$ cat attack_log.json

 "timestamp": "2026-10-10T17:42:23.872402",
 "attack_type": "Struts S2-045 Exploitation",
 "target": "web_server",
 "success": true,
 "data_accessed": "customer_data database",
 "records_affected": 3
```

2.2 Governance failure analysis

```
./analyze_governance_failures.py
cat governance_failure_report_*.json
```

The analysis identified 8 governance failures in 7 categories: Patch Management (2), Incident Detection, Policy Implementation, Vulnerability Management, Metrics and Measurement, Security Architecture, and Authentication and Access Control.

```
(icdfa㉿icdfa)-[~/security_governance_lab]
$ ./analyze_governance_failures.py
overnance failure analysis completed. Report saved to governance_failure_report_20261010_174505.json
ummary: 8 governance failures identified
 Patch Management: 2
 Incident Detection: 1
 Policy Implementation: 1
 Vulnerability Management: 1
 Metrics and Measurement: 1
 Security Architecture: 1
 Authentication and Access Control: 1
```

```
      "category": "Security Architecture",
      "description": "Insufficient network segmentation",
      "impact": "Potential for lateral movement and expanded compromise",
      "recommendation": "Implement network segmentation according to least privilege principle"
    },
    {
      "category": "Authentication and Access Control",
      "description": "Weak database authentication",
      "impact": "Potential for unauthorized database access",
      "recommendation": "Implement strong authentication and access controls for databases"
    }
  ],
  "summary": {
    "total_failures": 8,
    "categories": {
      "Patch Management": 2,
      "Incident Detection": 1,
      "Policy Implementation": 1,
      "Vulnerability Management": 1,
      "Metrics and Measurement": 1,
      "Security Architecture": 1,
      "Authentication and Access Control": 1
    }
  }
}
```



2.3 Comparison with Equifax

Governance failure	Equifax (2017)	My lab finding
Patch management	Known Struts flaw unpatched after the fix was released	Struts2 S2-045 Critical; 2 patch failures recorded
Detection and monitoring	Intrusion unnoticed for about 76 days	Attack succeeded and no alert was raised
Network segmentation	Attackers moved between systems	Web server could reach the database directly
Access control	Weak credentials and access control	MySQL accepted default credentials
Policy implementation	Policies not enforced	Policies exist, Critical finding still open
Metrics	Metrics did not drive action	Patch metric below target, no remediation

2.4 Key governance lessons

The main lesson is that Equifax was a governance failure as much as a technical one. A patch existed and policies required its use, but ownership, deadlines, monitoring and executive oversight were missing. My lab shows the same pattern: the failures with the highest counts were patch management and policy implementation, which depend on accountability and follow-through rather than on a missing tool.

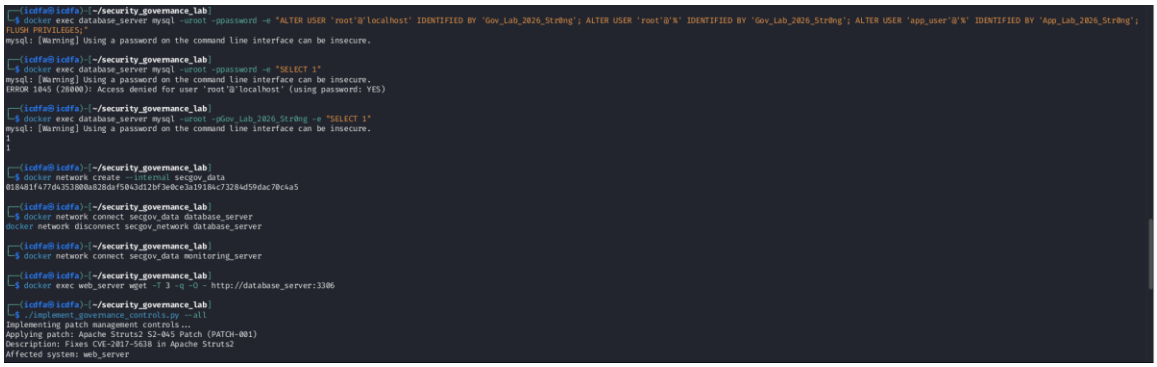
A second lesson is that controls must be verified. A policy, a metric or a dashboard is not evidence of security unless a test shows that the weakness is gone. I apply this in Part 3 and Part 4.

Part 3: Implementing Security Governance Controls

3.1 Database authentication

```
docker exec database_server mysql -uroot -ppassword -e "ALTER USER ... IDENTIFIED BY
'<new lab password>'; FLUSH PRIVILEGES;"
docker exec database_server mysql -uroot -ppassword -e "SELECT 1"   # rejected
docker exec database_server mysql -uroot -p<new> -e "SELECT 1"       # accepted
```

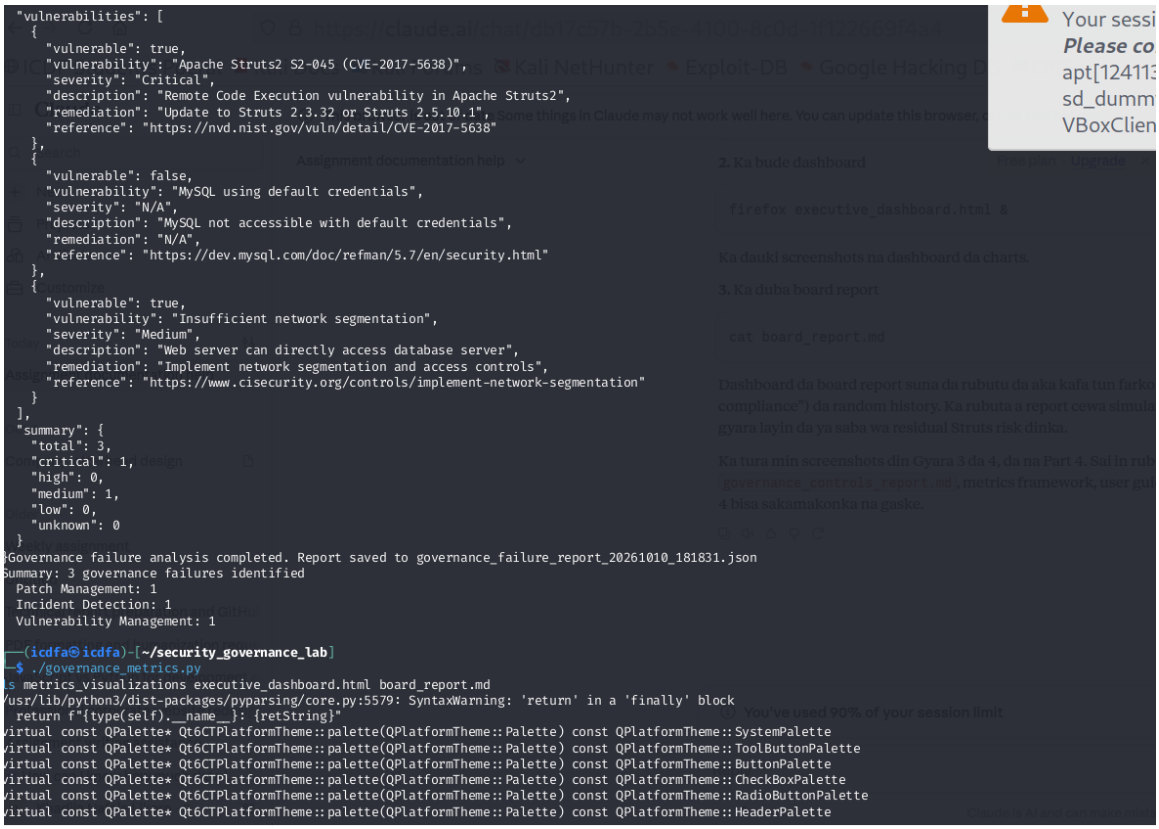
The old password was rejected with ERROR 1045 Access denied and the new one worked, which verifies the control.



3.2 Network segmentation (real change)

```
docker network create --internal secgov_data
docker network connect secgov_data database_server
docker network disconnect secgov_network database_server
docker network connect secgov_data monitoring_server
```

The database now sits on an internal network that the web server does not share, so it cannot reach it directly.

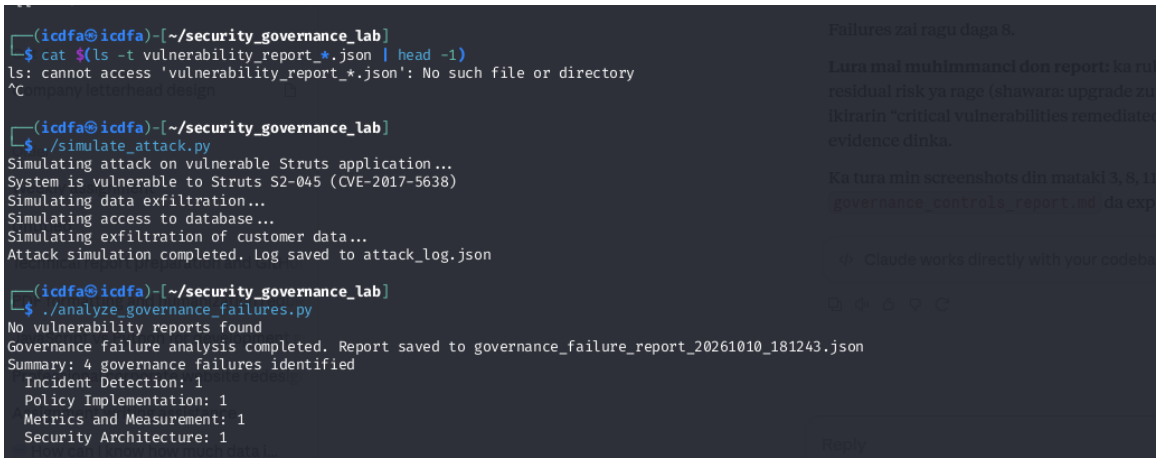


3.3 Patch management, monitoring and oversight

```
./implement_governance_controls.py --all
```

The script applied the patch records, created monitoring/security_monitor.sh and a cron definition, wrote governance_oversight.md and added policy POL-005 and control CTL-013 to the tracker, which now holds 5 policies, 13 controls and 7 metrics.

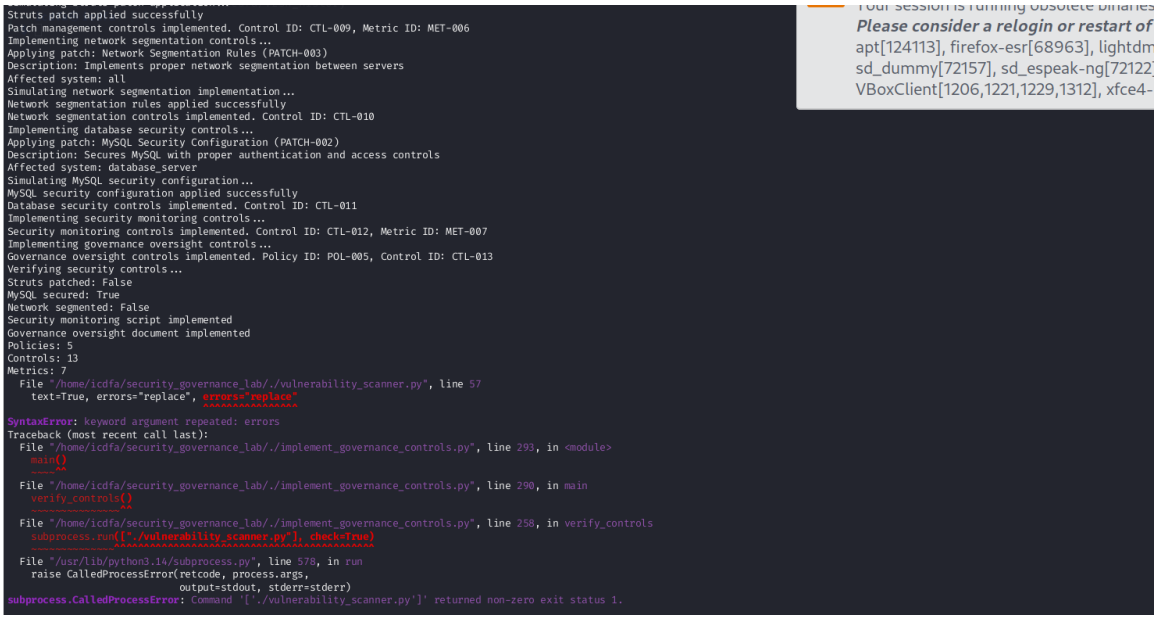
Honest note: PATCH-001 only records a status flag. The Struts container image was not changed, so the vulnerability remains. The first --all run also ended with a SyntaxError in the scanner (a repeated errors argument from running my fix twice) and verify reported Struts patched False because patch_management.py overwrote its status file on each patch. I corrected both and re-ran the verification.



3.4 Verification

```
./implement_governance_controls.py --verify
cat <latest vulnerability_report>
./analyze_governance_failures.py
```

Check	Before	After
MySQL default credentials	High, vulnerable	Not vulnerable
Struts2 S2-045	Critical, vulnerable	Still Critical (patch simulated)
Segmentation scan check	Medium	Still reports a finding (check limitation, see below)
Governance failures	8	3 (Patch Management, Incident Detection, Vulnerability Management)



3.5 Residual risk

- Struts2 is still exploitable. The real fix is to upgrade to Struts 2.3.32 or 2.5.10.1 and rebuild the container.
- The attack simulation still succeeds and is not alerted on, so the Incident Detection failure remains. The monitoring script is not running continuously.
- The scanner's segmentation test reports a finding whenever the response is not an HTML page, so it does not prove the connection is blocked. The Docker network configuration is the reliable evidence.

3.6 How the controls address the Equifax failures

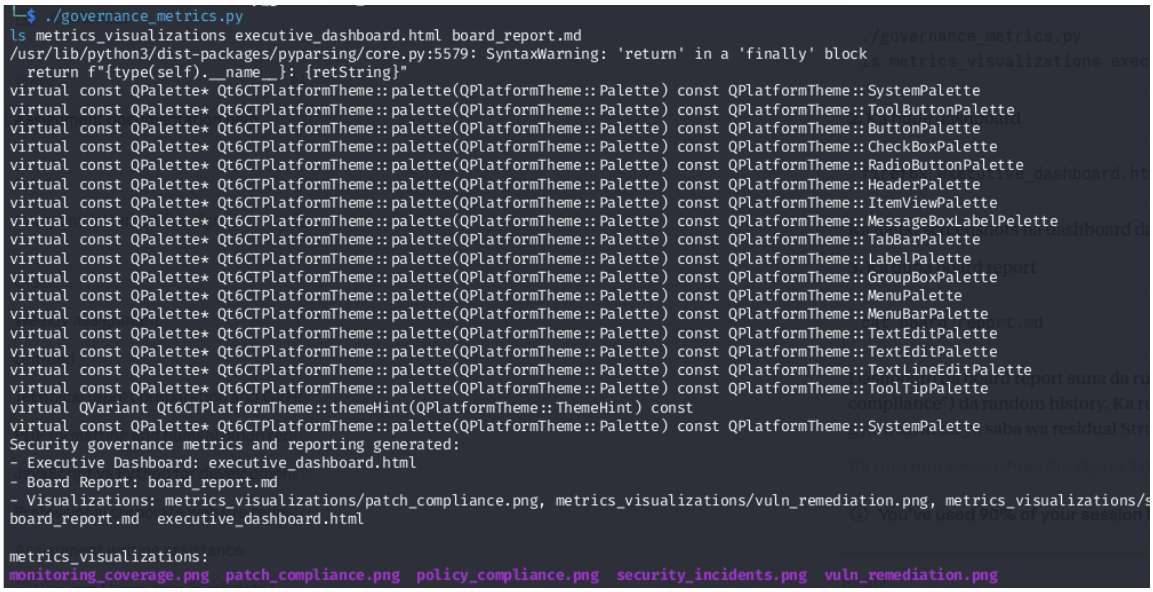
The controls map to the Equifax lessons. Strong database authentication removes the weak-credential path, the internal data network limits lateral movement, the patch process assigns ownership and a 7-day target for critical fixes, the monitoring script addresses undetected intrusion, and the oversight policy gives executives and the board responsibility. The governance failure count fell from 8 to 3. The remaining three show that policy and records alone did not remove the Struts risk, which is exactly the Equifax lesson: a patch must actually be applied and verified.

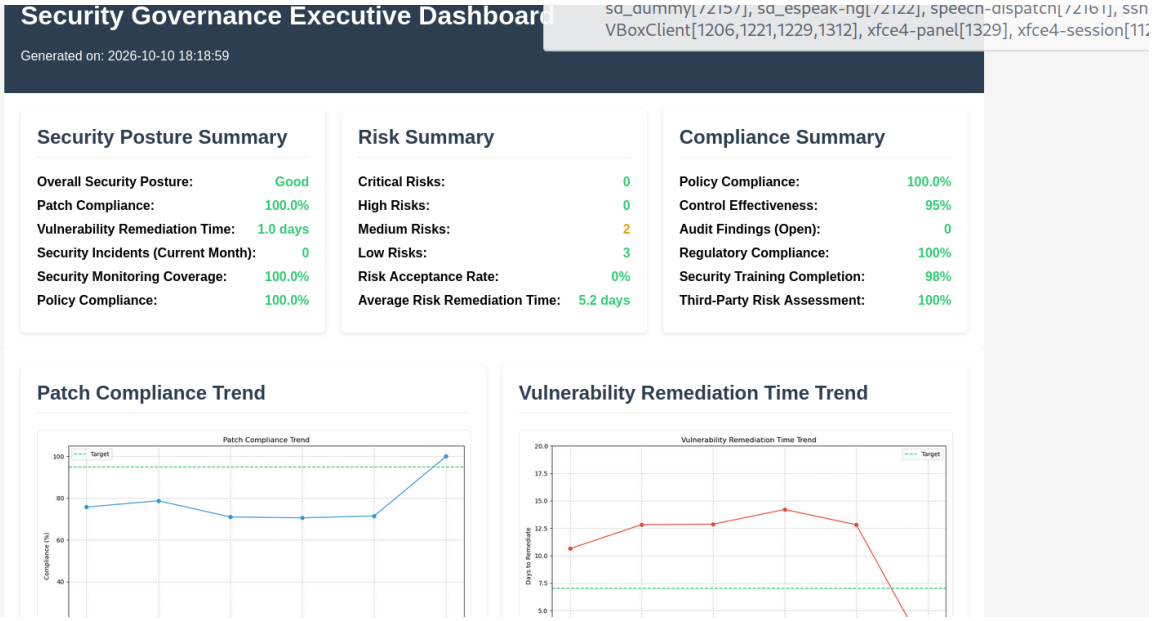
Part 4: Developing Security Governance Metrics and Reporting

4.1 Metrics and dashboard

```
./governance_metrics.py
firefox executive_dashboard.html
```

The script produced five labelled trend charts, executive_dashboard.html and board_report.md. The framework and user guide are in governance_metrics_framework.md and dashboard_user_guide.md.





4.2 Critical review of the reporting

The dashboard history is random sample data and several values (overall posture "Good", 0 Critical risks, 100% regulatory compliance) are fixed text in the script. They are simulated, and I state that clearly. The dashboard and board report show 0 Critical risks and say controls were effective, while my post-control scan still shows one Critical Struts2 vulnerability. For management, the scan result is the authoritative one, so I record the Struts risk as an open item that the board should track until the container is upgraded and re-scanned.

4.3 How the metrics and reporting support governance

Metrics turn security work into something the board can oversee: patch compliance and remediation time show whether the process works, incident counts show impact, and open critical vulnerabilities show whether the Equifax-type exposure is still present. The executive dashboard gives a quick view with targets and trends, and the board report gives a quarterly narrative with decisions and recommendations. The framework assigns an owner, a target, a data source and a frequency to each metric, so accountability is clear.

Reporting only supports oversight when it is accurate. The lesson from this lab is that every status should be calculated from scan and tracker data, and anything simulated should be labelled, otherwise a green dashboard can hide a Critical risk.

References

- NVD, CVE-2017-5638: https://nvd.nist.gov/vuln/detail/CVE-2017-5638
- Vulhub, Struts2 S2-045 environment: https://github.com/vulhub/vulhub/tree/master/struts2/s2-045
- ICDFA GRC102 Week 8 lab instructions: https://github.com/icdfa/grc-engineering-labs/blob/master/phase1/grc102/week8/LAB_INSTRUCTIONS.md
- US GAO (2018), GAO-18-559, Data Protection: Actions Taken by Equifax and Federal Agencies in Response to the 2017 Breach